IES INSTITUTO DE ESTUDIOS SUPERIORES

INGENIERÍA DE SOFTWARE I

UNIDAD I: “La ingeniería de Software”

PARTE 3

Ing. Fernando Alí
fali@iessantafe.edu.ar

ACTIVIDADES ESTRUCTURALES

Serie de actividades relacionadas que conducen a la elaboración de un producto (software)

Existen muchos diferentes PROCESOS DE SOFTWARE, pero todos deben incluir cuatro actividades que son fundamentales para la ISW

Especificación del Software	Diseño e implementación	Validación	Evolución del Software
Tienen que definirse tanto la funcionalidad del SW como las restricciones de su operación	Debe desarrollarse para cumplir con las especificaciones	Hay que validar el SW para asegurarse de que cumple lo que el cliente quiere	El SW tiene que evolucionar para satisfacer las necesidades cambiantes del sistema

ACTIVIDADES ESTRUCTURALES

CONCEPTOS GENERALES

-> Los Procesos de Software real son secuencias de actividades técnicas, colaborativas y administrativas.

-> La meta perseguida general es especificar, diseñar, implementar y probar un sistema de software.

-> Las cuatro actividades básicas de proceso (especificación, desarrollo, validación y evolución) se organizan de diversa manera en los diferentes procesos de desarrollo estudiados.

ESPECIFICACIÓN DEL SOFTWARE

CONCEPTOS GENERALES

-> También denominada INGENIERÍA DE REQUERIMIENTOS consiste en el proceso de comprender y definir qué servicios se requieren del sistema, así como también las restricciones de operación y desarrollo de sistema.

-> Es una etapa crítica ya que cometer errores en esta etapa conduce a inevitables problemas posteriores, principalmente durante las etapas de diseño y de implementación.

-> Este proceso se enfoca en producir un DOCUMENTO DE REQUERIMIENTOS que especifique los requerimientos de los interesados que debe cumplir el sistema a desarrollar.

ESPECIFICACIÓN DEL SOFTWARE

REQUERIMIENTOS

Se presentan en dos niveles de detalle:

-> Los USUARIOS FINALES Y CLIENTES necesitan un informe de requerimientos de alto nivel de detalle.

-> Los DESARROLLADORES DE SISTEMA precisan una descripción más detallada del sistema.

ESPECIFICACIÓN DEL SOFTWARE

ACTIVIDADES PRINCIPALES

1. ESTUDIO DE FACTIBILIDAD.

   -> Se realiza una estimación acerca de que si las necesidades identificadas del usuario se cubren con las actuales tecnologías de software y hardware.

   -> En el estudio se considera si el sistema propuesto tendra un costo-beneficio desde el punto de vista empresarial.

   -> Se pregunta si el sistema puede desarrollarse dentro de las restricciones presupuestales existentes.

   -> El estudio debe ser rápido y relativamente barato.

   -> El resultado debe informar la decisión respecto a si se continúa o no con un análisis más detallado.

ESPECIFICACIÓN DEL SOFTWARE

ACTIVIDADES PRINCIPALES

2. OBTENCIÓN Y ANÁLISIS DE REQUERIMIENTOS

-> Proceso de derivar los requerimientos del sistema mediante observación de los sistemas existentes, discusión con los usuarios y los potenciales proveedores.

-> Puede incluir el desarrollo de uno o más prototipos para ayudar a entender el sistema que se va a especificar.

ESPECIFICACIÓN DEL SOFTWARE

ACTIVIDADES PRINCIPALES

3. ESPECIFICACIÓN DE REQUERIMIENTOS

-> Consiste en transcribir la información recopilada durante la actividad de análisis, en un documento que define el conjunto de requerimientos.

-> Se incluyen dos clases de requerimientos:

- Del usuario: informes abstractos de requerimientos del sistema para el cliente o usuario final.

- Del sistema: descripción detallada de las funcionalidades a ofrecer-

ESPECIFICACIÓN DEL SOFTWARE

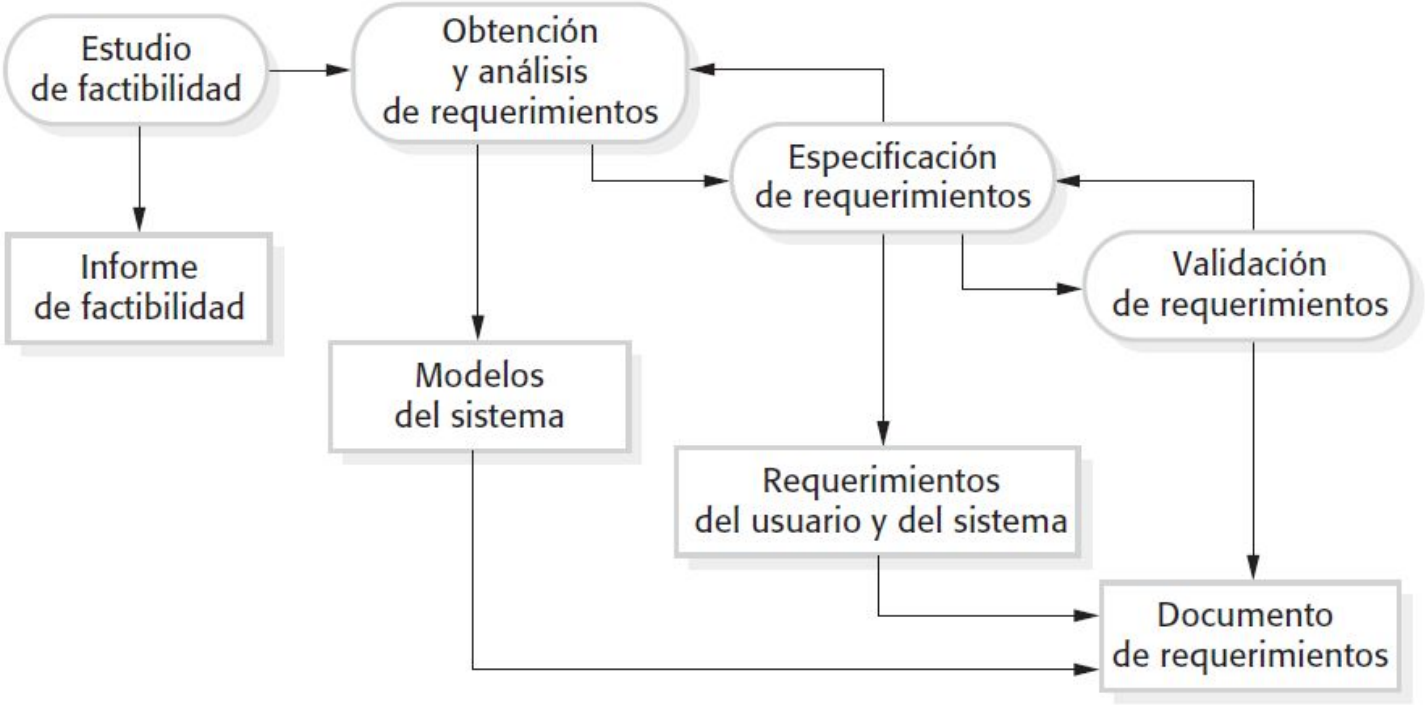
ACTIVIDADES PRINCIPALES

4. VALIDACIÓN DE REQUERIMIENTOS

-> Verifica que los requerimientos sean realistas, coherentes y completos.

-> Se descubren errores en el Documento de Requerimientos, que deben modificarse con la finalidad de corregir dichos problemas.

ESPECIFICACIÓN DEL SOFTWARE



DISEÑO E IMPLEMENTACIÓN DEL SOFTWARE

CONCEPTOS GENERALES

-> Consiste en el proceso de convertir una especificación del sistema en un sistema ejecutable.

-> Incluye procesos de diseño y de programación, aunque también puede incluir la corrección de especificaciones.

-> Un DISEÑO es una descripción de la estructura del software que se va a implementar, los modelos y estructuras de datos utilizados por el sistema.

-> No se llega a una creación terminada sino que se desarrolla el diseño de manera iterativa. Se agrega formalidad y detalle conforme se realiza el diseño con backtracking constante para corregir diseños anteriores.

-> Las actividades en el proceso de diseño varían dependiendo del tipo de sistema a desarrollar.

DISEÑO E IMPLEMENTACIÓN DEL SOFTWARE

ACTIVIDADES PRINCIPALES

1. DISEÑO ARQUITECTÓNICO

-> Se identifica la estructura global del sistema, los principales componentes (llamados subsistemas o módulos), sus relaciones y cómo se distribuyen.

2. DISEÑO DE INTERFAZ

-> Se definen las interfaces entre los componentes del sistema. Con una interfaz precisa, es factible usar un componente sin que otros tengan que saber cómo se implementó. Una vez que se acuerdan las especificaciones de interfaz, los componentes se diseñan y se desarrollan de manera concurrente.

DISEÑO E IMPLEMENTACIÓN DEL SOFTWARE

ACTIVIDADES PRINCIPALES

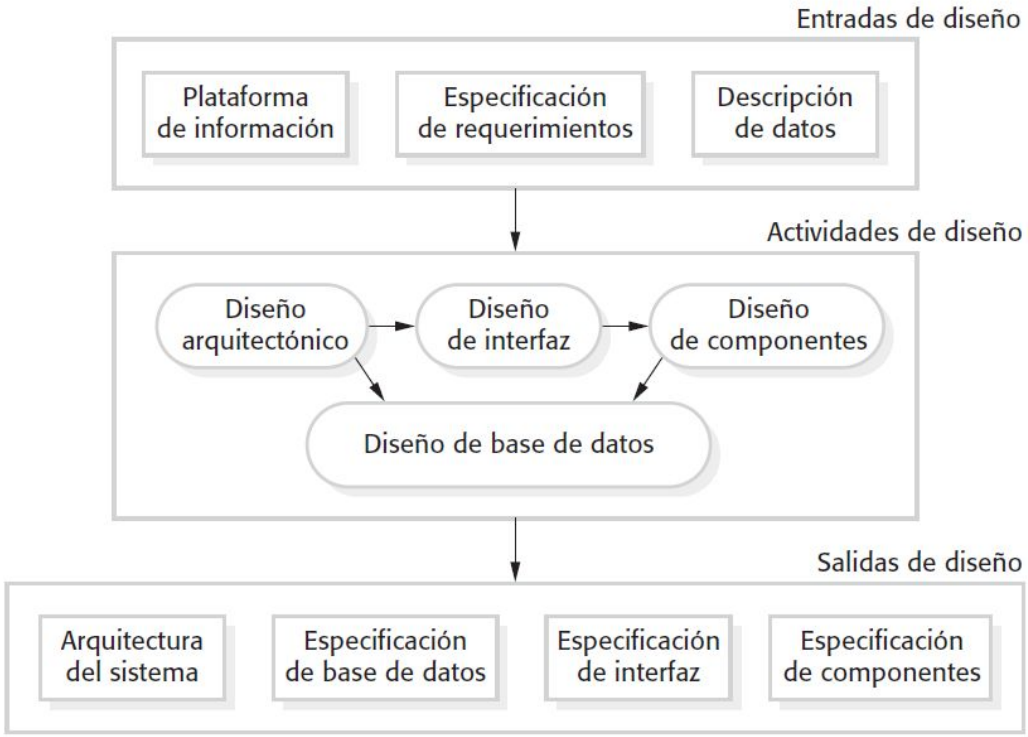
3. DISEÑO DE COMPONENTES

-> Se toma cada componente del sistema y se diseña cómo funcionará. Esto puede ser un simple dato de la funcionalidad que se espera implementar, y al programador se le deja el diseño específico.

4. DISEÑO DE BASE DE DATOS

-> Se diseñan las estructuras de sistema de datos y cómo se representarán en una base de datos.

DISEÑO E IMPLEMENTACIÓN DEL SOFTWARE



VALIDACIÓN DEL SOFTWARE

CONCEPTOS GENERALES

-> Se usa para mostrar que un sistema cumple tanto con sus especificaciones como con las expectativas del cliente.

-> Las pruebas del programa, donde el sistema se ejecuta a través de datos de prueba simulados, constituye la principal técnica de validación.

-> También se pueden incluir procesos de comprobación, como inspecciones y revisiones en cada etapa del proceso de software.

-> Los sistemas no deben ponerse a prueba como una unidad monolítica. El proceso de prueba se desarrolla en tres etapas: se ponen a prueba los COMPONENTES DEL SISTEMA, luego, se hace lo mismo con el SISTEMA INTEGRADO, y finalmente, es sistema se pone a prueba con los CLIENTES.

VALIDACIÓN DEL SOFTWARE

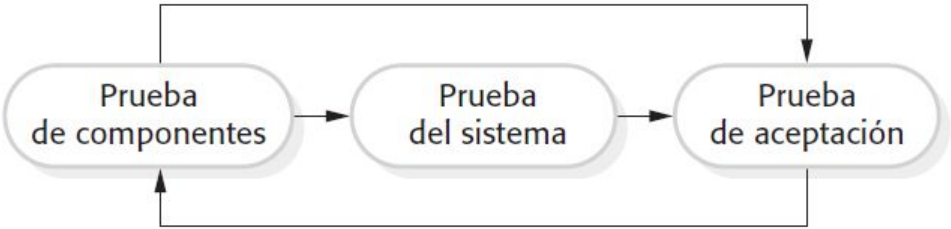
ACTIVIDADES PRINCIPALES

1. PRUEBA DE DESARROLLO

-> Se ponen a prueba los componentes que constituyen el sistema.

-> Cada componente se pone a prueba de forma independiente, es decir, sin otros componentes del sistema.

-> Pueden ser simples entidades o agrupamientos coherentes de dichas entidades.



VALIDACIÓN DEL SOFTWARE

ACTIVIDADES PRINCIPALES

2. PRUEBAS DEL SISTEMA

-> Los componentes del sistema se integran para crear un sistema completo.

-> Este proceso tiene la finalidad de descubrir errores que resultan de interacciones no anticipadas de componentes y problemas de interfaz de componente así como de mostrar que el sistema cubre su requerimientos funcionales y no funcionales.

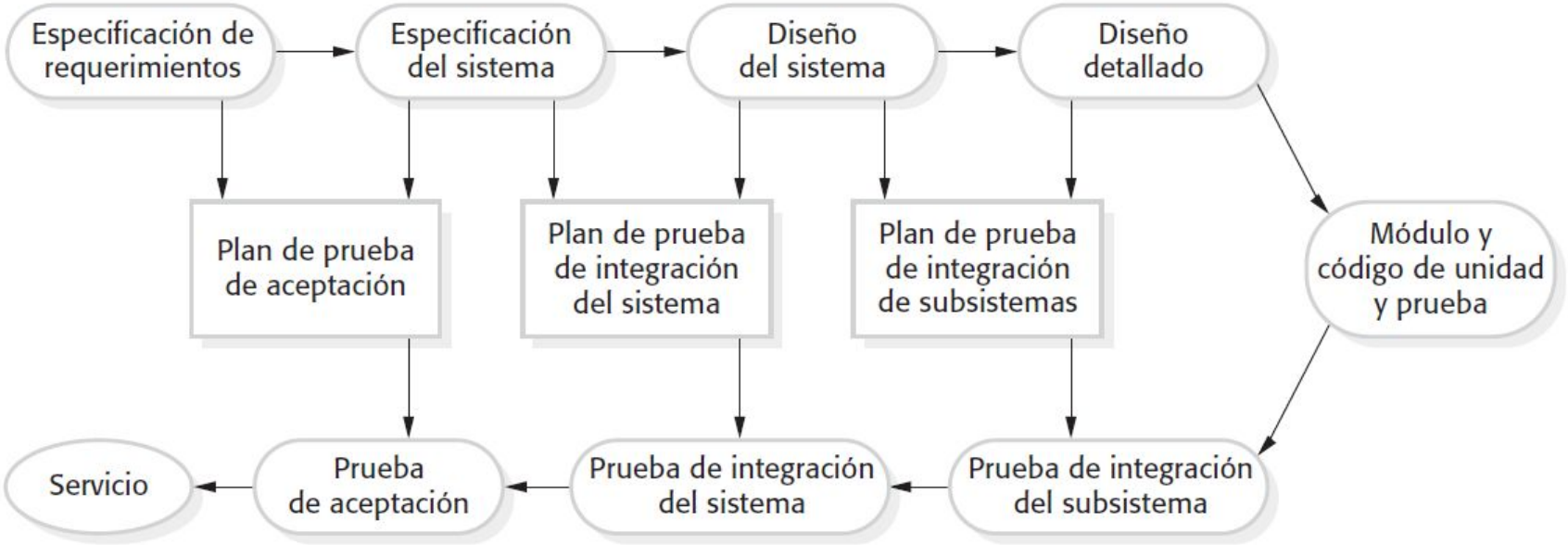
3. PRUEBAS DE ACEPTACIÓN

-> Etapa final del proceso de pruebas, antes que el sistema se acepte para uso operacional.

-> Se pone a prueba con datos suministrados por el cliente del sistema, en vez de los de prueba simulados.

-> Revelan errores y omisiones de la definición de requerimientos.

VALIDACIÓN DEL SOFTWARE



VALIDACIÓN DEL SOFTWARE

-> Los programadores construyen sus propios datos de prueba y experimentan el código de manera incremental conforme lo desarrollan.

-> Si se usa un enfoque incremental para el desarrollo, cada incremento debe ponerse a prueba conforme se diseña, y tales pruebas se basan en los requerimientos para dicho incremento.

EVOLUCIÓN DEL SOFTWARE

CONCEPTOS GENERALES

-> La flexibilización de los sistemas es una de las razones principales por las que cada vez más software se incorpora en los sistemas grandes y complejos.

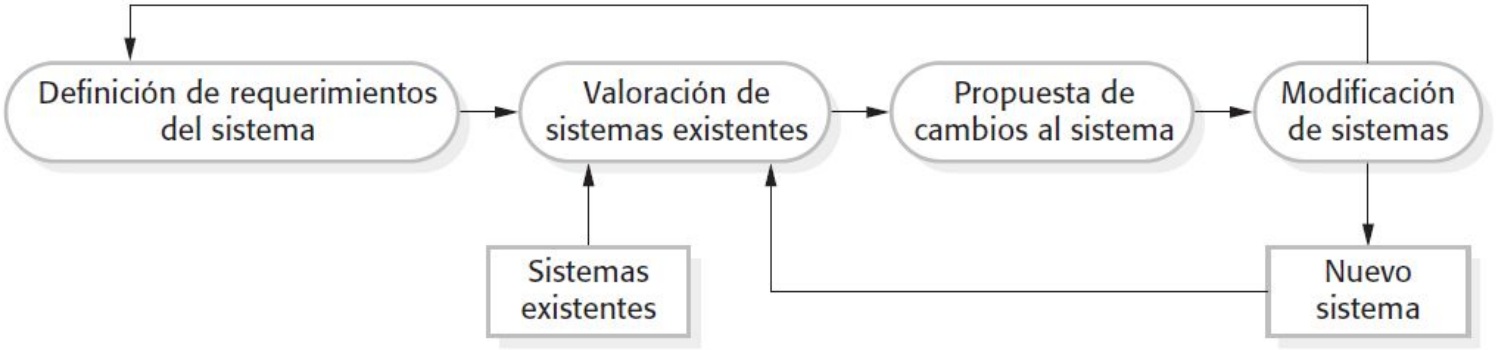
-> En cualquier momento, durante o después del desarrollo del sistema, pueden hacerse cambios al software.

-> Siempre ha habido división entre el proceso de desarrollo de software y el proceso de evolución (o mantenimiento).

-> Muchas personas consideran el desarrollo de software como una actividad creativa , en la cual se diseña el sistema desde un concepto inicial. No obstante consideran en ocasiones el mantenimiento como algo insulso o poco interesante.

-> En lugar de dos procesos separados, es más realista pensar en la ISW como un proceso evolutivo, donde el SW cambia continuamente a lo largo de su vida, en función de los requerimientos y las necesidades cambiantes del cliente.

EVOLUCIÓN DEL SOFTWARE



ENFRENTAR EL CAMBIO

- El cambio es inevitable en todos los grandes proyectos de software.
- Los requerimientos varían conforme la empresa debe responder a los a presiones externas o modifican las prioridades y exigencias administrativas.
- A medida que se ponen a disposición nuevas tecnologías, surgen nuevas posibilidades de diseño e implementación.
- Cualquiera sea el modelo de proceso de software utilizado, es esencial que se ajusten cambios al SW a desarrollar.
- El cambio se agrega a los costos del desarrollo de software debido a que significa más trabajo terminado que debe volver a realizarse.

ENFRENTAR EL CAMBIO

- Existen dos enfoques relacionados que se usan para REDUCIR LOS COSTOS DE REHACER:
  - Evitar el cambio, donde el proceso de software incluye actividades que anticipan cambios posibles antes de requerirse la labor de rehacer.
  - Tolerancia al cambio, donde el proceso se diseña de modo que los cambios se ajusten con un costo relativamente bajo.

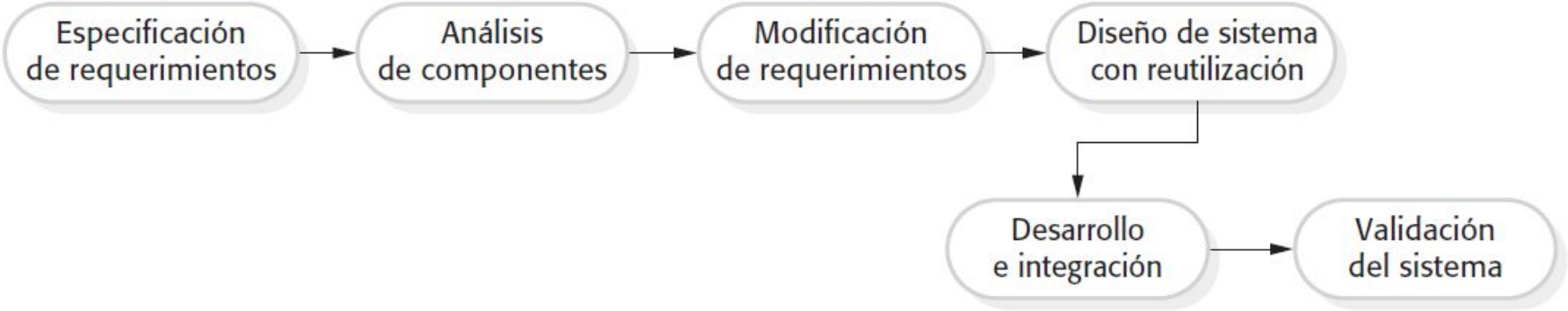
ISW ORIENTADA A LA REUTILIZACIÓN

- En la mayoría de los proyectos suele aparecer la REUTILIZACIÓN DE SISTEMAS.
- Sucede cuando las personas implicadas en un proyecto conocen o ya han trabajado en un DISEÑO o CÓDIGO similar.
- Se suele utilizar y modificar los sistemas o partes de sistemas existentes de acuerdo a las nuevas necesidades y se los incorpora a los nuevos sistemas.
- Esta acción “informal” ocurre independientemente del PROCESO DE DESARROLLO DE SOFTWARE que se esté utilizando.

ISW ORIENTADA A LA REUTILIZACIÓN

- Los enfoques orientados a la REUTILIZACIÓN DE SOFTWARE se apoyan en la base de los COMPONENTES de SOFTWARE REUTILIZABLES y en la integración y composición de dichos componentes.
- Analizando el PROCESO DE DESARROLLO ORIENTADO A LA REUTILIZACIÓN, aunque la etapa de especificación de requerimientos y de validación se parecen a otros procesos, las actividades intermedias son diferentes.

ISW ORIENTADA A LA REUTILIZACIÓN



ISW ORIENTADA A LA REUTILIZACIÓN

1. ANÁLISIS DE COMPONENTES.

   Dada la especificación de requerimientos, se realiza una búsqueda de componentes para implementar dicha especificación. Por lo general no hay coincidencia exacta pero lo componentes usados brindan parte de la funcionalidad.

2. MODIFICACIÓN DE REQUERIMIENTOS.

   Se utiliza la información de los componentes descubiertos. Luego, se modifican para hacerlos funcionales al diseño del sistema en desarrollo. Cuando esto no es posible se buscan soluciones alternativas.

ISW ORIENTADA A LA REUTILIZACIÓN

3. DISEÑO DE SISTEMA CON REUTILIZACIÓN

   Se diseña el marco conceptual del sistema o se reutiliza uno existente. Los creadores toman en cuenta los componentes a reutilizar y se organiza el marco de referencia en base a ello. Generalmente, es necesario diseñar “algo” de software nuevo en caso de que algunos componentes no se encuentren disponibles.

4. DESARROLLO O INTEGRACIÓN

   Se diseña la parte del software que no se consigue de forma externa (a través de otros componentes existentes) y se integra al resto de los componentes para crear el nuevo sistema. Es usual que el proceso de integración en estos casos sea parte del proceso de desarrollo en lugar de una actividad diferente o independiente.

ISW ORIENTADA A LA REUTILIZACIÓN

Existen tres TIPOS DE COMPONENTES DE SOFTWARE que pueden usarse en un proceso orientado a la reutilización:

1. SERVICIOS WEB que se desarrollan en concordancia para atender servicios estándares y que están disponibles para la invocación remota.
2. COLECCIÓN DE OBJETOS que se desarrollan como un paquete para su integración con un marco de componentes.
3. SISTEMAS DE SOFTWARE INDEPENDIENTES que se configuran para usar en un entorno particular.

ISW ORIENTADA A LA REUTILIZACIÓN

VENTAJAS

- Reduce la cantidad de software a desarrollar
- Reduce los tiempos de desarrollo
- Reduce los costos de desarrollo
- Disminuyen los riesgos
- Conduce a entregas más rápidas

DESVENTAJAS

- Puede conducir a un sistema que no cumpla con los compromisos específicos de los requisitos.
- Se pierde el control sobre la evolución del sistema que queda sujeto a las nuevas versiones de los componentes que no están bajo el control de la organización que los utiliza.